```
 1
 2  // COS30008 - Problem Set 3, 2026
 3
 4  #pragma once
 5
 6  #include "SortablePair.h"
 7
 8  #include <optional>
 9  #include <cassert>
10  #include <algorithm>
11
12  template<typename T, typename P>
13  class PriorityQueue
14  {
15  public:
16
17      using value_type = SortablePair<P, T>;
18
19      PriorityQueue() noexcept;
20
21      ~PriorityQueue() noexcept;
22
23      PriorityQueue( const PriorityQueue& ) = delete;
24      PriorityQueue& operator=( const PriorityQueue& ) = delete;
25
26      size_t count() const noexcept;
27      size_t capacity() const noexcept;
28
29      std::optional<T> top() const noexcept;
30      void push( const T& aValue, const P& aPriority ) noexcept;
31      void pop() noexcept;
32
33  private:
34
35      value_type* fElements;
36      size_t fHead;
37      size_t fTail;
38      size_t fCapacity;
39
40      void sort() noexcept
41      {
42          std::sort( &fElements[fHead], &fElements[fTail] );
43      }
44
45      void resize( size_t aCapacity );
46      void ensure_capacity();
47      void adjust_capacity();
48  };
49
```

```
50  template<typename T, typename P>
51  PriorityQueue<T, P>::PriorityQueue() noexcept
52      : fElements(static_cast<value_type*>(::operator new(sizeof
          (value_type)))),
53      fHead(0),
54      fTail(0),
55      fCapacity(1)
56  {
57  }
58
59  template<typename T, typename P>
60  PriorityQueue<T, P>::~PriorityQueue() noexcept
61  {
62      for (size_t i = fHead; i < fTail; i++)
63      {
64          fElements[i].~value_type();
65      }
66      ::operator delete(fElements);
67  }
68
69  template<typename T, typename P>
70  size_t PriorityQueue<T, P>::count() const noexcept
71  {
72      return fTail - fHead;
73  }
74
75  template<typename T, typename P>
76  size_t PriorityQueue<T, P>::capacity() const noexcept
77  {
78      return fCapacity;
79  }
80
81  template<typename T, typename P>
82  std::optional<T> PriorityQueue<T, P>::top() const noexcept
83  {
84      if (count() == 0)
85      {
86          return std::nullopt;
87      }
88      return fElements[fHead].second();
89  }
90
91  template<typename T, typename P>
92  void PriorityQueue<T, P>::resize(size_t aCapacity)
93  {
94      value_type* lNewElements = static_cast<value_type*>(::operator new
          (aCapacity * sizeof(value_type)));
95
96      size_t lCount = count();
```

```
 97
 98      for (size_t i = 0; i < lCount; i++)
 99      {
100          new (&lNewElements[i]) value_type(fElements[i + fHead]);
101          fElements[i + fHead].~value_type();
102      }
103
104      ::operator delete(fElements);
105
106      fElements = lNewElements;
107      fHead = 0;
108      fTail = lCount;
109      fCapacity = aCapacity;
110  }
111
112  template<typename T, typename P>
113  void PriorityQueue<T, P>::ensure_capacity()
114  {
115      if (fTail == fCapacity)
116      {
117          resize(fCapacity * 2);
118      }
119  }
120
121  template<typename T, typename P>
122  void PriorityQueue<T, P>::adjust_capacity()
123  {
124      if (fCapacity > 1 && count() * 4 <= fCapacity)
125      {
126          size_t lNewCapacity = fCapacity / 2;
127          if (lNewCapacity < 1) lNewCapacity = 1;
128          resize(lNewCapacity);
129      }
130  }
131
132  template<typename T, typename P>
133  void PriorityQueue<T, P>::push(const T& aValue, const P& aPriority)
       noexcept
134  {
135      ensure_capacity();
136      new (&fElements[fTail++]) value_type(aPriority, aValue);
137      std::stable_sort(&fElements[fHead], &fElements[fTail]);
138  }
139
140  template<typename T, typename P>
141  void PriorityQueue<T, P>::pop() noexcept
142  {
143      assert(count() > 0);
144      fElements[fHead].~value_type();
```

```
145         fHead++;
146         adjust_capacity();
147     }
```